

Course Pack 01

Mathematical Foundations and Laboratory Setup Quantum Entanglement & Quantum Computing (UE23EC343AB3)

PES University

What Is In This Pack

This single document covers the first class and the first lab.

Part I builds the mathematics: sets and maps, groups, the hierarchy of spaces, and the passage from linear maps to matrices. It ends by proving that every reversible classical computation is a special case of a quantum computation.

Part II gets Qiskit and Cirq running on your machine, and verifies the install by checking the theorems from Part I numerically. Setup labs are usually dead time. This one is not.

One accompanying file: `QEQC_Class01_Lab00.ipynb`

The notebook is both the live version of Part I and the whole of the Lab 00 verification. There are no separate scripts to download and no separate thing to run.

Quote to Start On

“God does not play dice with the universe. He plays an odd kind of poker, and He’s card-counting.” — overheard at a physics conference, attribution disputed

Contents

I	Mathematical Foundations	3
1	Why Start Here? Reversibility	3
2	Sets, Products, and Maps	4
2.1	Sets	4
2.2	Maps	4
3	Groups	5
3.1	Four Observations Before the Definition	5
3.2	The Examples That Matter	7
3.3	Order of an Element	8

4	Vector Spaces over $\mathbb{C}$	9
4.1	The Space We Actually Care About	9
4.2	Span, Independence, Basis, Dimension	10
4.3	Inner Product	11
5	The Hierarchy of Spaces	11
5.1	The Four Layers	12
5.2	The Picture	13
5.3	Every Inclusion Is Strict	14
5.4	Completeness and Hilbert Space	15
5.5	Where Each Layer Shows Up in Quantum Mechanics	17
6	From Linear Maps to Matrices	17
6.1	Closing the Loop	18
7	Problem Set	19
8	Reading and What Comes Next	20
II	Laboratory Setup: Qiskit and Cirq	21
9	What We Are Installing, and Why Two of Them	21
10	Part A: Install Python 3.12	22
11	Part B: Build the Two Environments	22
12	Part C: Verify with the Notebook	24
12.1	The Sign-Off Cell	24
12.2	What Is in the Notebook	26
13	The Same Circuit in Both SDKs	26
14	Optional: Real Hardware via IBM Quantum	27
15	Troubleshooting	29
16	Before You Leave	30

Part I

Mathematical Foundations

The One Sentence This Entire Class Is About

Classical computation is a bijection on a finite set.

Quantum computation is a unitary map on a complex vector space.

Every definition in these notes exists to make one half of that sentence precise. Sets and maps build the left half. Groups tell us the structure that reversible operations carry. Vector spaces and matrices build the right half. By the last page we will prove that the left half sits *inside* the right half, and the gap between them is the entire rest of this course.

1 Why Start Here? Reversibility

Consider the classical **AND** gate. Feed it the inputs $(0, 1)$ and it outputs 0.

Now run the question backwards. You are told the output is 0. What were the inputs? There are three possibilities: $(0, 0)$, $(0, 1)$, $(1, 0)$. You cannot tell which. **Information has been destroyed.**

Compare with the **NOT** gate. Told the output, you know the input exactly. Nothing is lost.

Landauer's Principle (1961)

Erasing one bit of information in a computation dissipates at least

$$\Delta Q \geq k_B T \ln 2$$

of heat into the environment. Irreversibility is not merely inelegant. It has a thermodynamic price tag.

Quantum mechanics goes further. The evolution of an isolated quantum system is governed by the Schrödinger equation, whose solution $U(t) = e^{-iHt/\hbar}$ is always invertible. There is no room in the formalism for an operation that forgets. **Every quantum gate is reversible.**

So the objects we care about are the *reversible* operations. Our first job is to say precisely what that means, and our second job is to discover what structure the collection of all such operations carries.

$$\text{reversible} = \text{invertible} = \text{bijective}$$

2 Sets, Products, and Maps

2.1 Sets

Set, Membership, Subset

A **set** S is a collection of distinct objects, called its **elements**. We write $x \in S$ if x is an element of S , and $A \subseteq B$ if every element of A is also an element of B . The number of elements of a finite set S is its **cardinality**, written $|S|$.

We will not do axiomatic set theory. For this course a set is simply a bag of labels, and the only operation we truly need is the following one.

Cartesian Product

The **Cartesian product** of sets A and B is the set of ordered pairs

$$A \times B = \{ (a, b) : a \in A, b \in B \}.$$

Its cardinality is $|A \times B| = |A| \cdot |B|$.

Writing $B = \{0, 1\}$ for the set of bit values, the n -fold product

$$B^n = \underbrace{B \times B \times \cdots \times B}_{n \text{ times}}$$

is the set of all n -bit strings, and $|B^n| = 2^n$.

The Sentence to Remember from This Subsection

The state of a classical n -bit register is a **single element** of B^n . One point, chosen out of 2^n available points.

A quantum register of n qubits will instead be a **vector** in a 2^n -dimensional complex vector space. The distance between “one point in a set of size 2^n ” and “a vector in a space of dimension 2^n ” is the whole subject you have enrolled in.

2.2 Maps

Map and its Types

A **map** (or function) $f : A \rightarrow B$ assigns to each $a \in A$ exactly one element $f(a) \in B$.

- f is **injective** (one-to-one) if $f(a_1) = f(a_2) \implies a_1 = a_2$. Distinct inputs never collide.
- f is **surjective** (onto) if every $b \in B$ equals $f(a)$ for some $a \in A$.
- f is **bijective** if it is both. Equivalently, an inverse map $f^{-1} : B \rightarrow A$ exists.

Example: AND is Not Reversible, CNOT Is

(a) **The AND gate.** $\text{AND} : B^2 \rightarrow B$, with $\text{AND}(a, b) = a \wedge b$.

input	output
00	0
01	0
10	0
11	1

Three inputs map to the same output, so AND is not injective, hence not bijective, hence not reversible. **AND cannot be a quantum gate.** (We will see later that it can be *embedded* inside a larger reversible gate, but that costs extra qubits.)

(b) **The CNOT gate.** $\text{CNOT} : B^2 \rightarrow B^2$, with $(a, b) \mapsto (a, a \oplus b)$, where $\oplus$ is addition modulo 2.

input	output
00	00
01	01
10	11
11	10

Every element of B^2 appears exactly once in the output column. CNOT is a bijection. Concretely, it swaps the labels 10 and 11 and leaves 00 and 01 alone.

CNOT is a permutation of a four-element set. Nothing more mysterious than that, at this stage. When it returns in Unit 3 as the gate that manufactures entanglement, remember that you first met it as a shuffle of four labels.

Summary of Section 2

Reversible classical computation on n bits = a bijection $B^n \rightarrow B^n$ = a permutation of 2^n objects.

3 Groups

We now have a supply of reversible operations. The natural next question is: what does the *collection of all of them* look like?

3.1 Four Observations Before the Definition

Fix the set B^2 and consider all bijections $B^2 \rightarrow B^2$, that is, all shuffles of four labels. Four facts are immediate.

1. Performing one shuffle and then another gives a shuffle.
2. When composing three shuffles, the bracketing does not matter: $(fg)h = f(gh)$.
3. “Do nothing” is a shuffle.

4. Every shuffle can be undone by another shuffle.

Those four sentences are the axioms of a group. The abstraction is not imposed from outside; it is read off from an example you already understand.

Group

A **group** is a set G together with a binary operation $\cdot : G \times G \rightarrow G$ satisfying:

G1. Closure. For all $a, b \in G$, the product $a \cdot b$ lies in G .

G2. Associativity. $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ for all $a, b, c \in G$.

G3. Identity. There exists $e \in G$ with $e \cdot a = a \cdot e = a$ for all a .

G4. Inverse. For every $a \in G$ there exists $a^{-1} \in G$ with $a \cdot a^{-1} = a^{-1} \cdot a = e$.

If in addition $a \cdot b = b \cdot a$ for all $a, b \in G$, the group is called **abelian** (or commutative).

A Group Is Not Just a Set

A group is a set *together with* an operation. The same underlying set can carry several different group structures, or none at all.

For instance $\{0, 1\}$ is a group under XOR (identity 0, each element its own inverse), but $\{0, 1\}$ under ordinary multiplication is *not* a group, because 0 has no inverse.

Whenever you are asked “is this a group?”, your first question back should be “under which operation?”

3.2 The Examples That Matter

Five Groups You Will Meet Again

1. $(\mathbb{Z}, +)$. Identity 0, inverse of n is $-n$. Infinite, abelian. A warm-up.
2. $(\mathbb{Z}_2, \oplus)$, **the group with two elements**. The set $\{0, 1\}$ under XOR. This *is* the classical NOT gate viewed as a group: applying NOT twice returns you to where you started. In matrix language,

$$X^2 = I.$$

3. S_3 , **the permutations of three objects**. Six elements. Let σ swap positions 1 and 2, and let τ swap positions 2 and 3. Applying σ then τ sends $(1, 2, 3) \mapsto (2, 3, 1)$, whereas τ then σ sends $(1, 2, 3) \mapsto (3, 1, 2)$. These are different, so

$$\sigma\tau \neq \tau\sigma.$$

S_3 is **non-abelian**. Read the next box before moving on.

4. $U(1) = \{e^{i\theta} : \theta \in [0, 2\pi)\}$ **under multiplication**. The unit circle in $\mathbb{C}$. Infinite, continuous, abelian. This group will reappear as **global phase**, and we will later show that global phase is physically unobservable, meaning the true space of qubit states is what remains after this entire group has been divided out.

5. **The Pauli group $\mathcal{P}_1$** . Start from the four matrices

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Multiply two of them and see what happens (do this yourself before reading on):

$$XZ = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = -iY, \quad ZX = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = iY.$$

Two consequences. First, $XZ = -ZX$: these matrices **anticommute**. Second, and structurally more important, the result $-iY$ is *not* in the set $\{I, X, Y, Z\}$. That set fails closure, so it is not a group.

The fix is to admit the phases that were forced on us. The genuine Pauli group is

$$\mathcal{P}_1 = \{\pm 1, \pm i\} \cdot \{I, X, Y, Z\},$$

a group of 16 elements. Closure was not a bureaucratic axiom; it told us something was missing and told us exactly what.

Why Non-Commutativity Deserves Your Attention

In S_3 we found $\sigma\tau \neq \tau\sigma$, and among the Paulis we found $XZ = -ZX$. This is not a technical annoyance. It is the mathematical root of two of the most famous features of quantum mechanics:

- The **order of gates in a circuit matters**. HZ and ZH are different physical operations.
- Observables that fail to commute cannot be simultaneously sharp. This is where the **uncertainty principle** comes from, via $[\sigma_i, \sigma_j] = 2i\epsilon_{ijk}\sigma_k$, which you will meet in Unit 2.

If quantum theory were built on an abelian group, there would be no uncertainty principle and very little to talk about.

Overheard in Office Hours

“Non-abelian just means the universe cares what order you do things in. Your laundry already knew this: socks first, then shoes, not the other way around.”

3.3 Order of an Element

Order

The **order** of $g \in G$, written $\text{ord}(g)$, is the smallest positive integer k such that $g^k = e$. If no such k exists, g has infinite order.

For quantum gates this is a concrete and useful number: it tells you how many times you must repeat a gate before the circuit returns to the identity.

Gate	Matrix	Order
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	2
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	2
H	$\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	2
S	$\text{diag}(1, i)$	4
T	$\text{diag}(1, e^{i\pi/4})$	8

Notice the arithmetic hiding in the last two rows:

$$T^2 = S, \quad S^2 = Z, \quad Z^2 = I.$$

The standard gate set of a quantum computer is not a random pile of matrices. It has structure, and that structure is group structure.

Scope Note

Cosets, Lagrange's theorem, normal subgroups, quotient groups and the homomorphism theorems are all standard next steps in a group theory course, and none of them is needed for this syllabus. They return in a serious way only if you study the *stabilizer formalism* for quantum error correction. If you want that story now, read Nielsen & Chuang Chapter 10.

4 Vector Spaces over $\mathbb{C}$

Groups told us what the legal operations are. Vector spaces tell us what those operations act *on*.

Complex Vector Space

A **complex vector space** V is a set of objects called vectors, equipped with addition $V \times V \rightarrow V$ and scalar multiplication $\mathbb{C} \times V \rightarrow V$, such that for all $|u\rangle, |v\rangle, |w\rangle \in V$ and $\alpha, \beta \in \mathbb{C}$:

$$\begin{array}{ll} |u\rangle + |v\rangle = |v\rangle + |u\rangle & \alpha(|u\rangle + |v\rangle) = \alpha|u\rangle + \alpha|v\rangle \\ (|u\rangle + |v\rangle) + |w\rangle = |u\rangle + (|v\rangle + |w\rangle) & (\alpha + \beta)|v\rangle = \alpha|v\rangle + \beta|v\rangle \\ \text{there is } |0\rangle \text{ with } |v\rangle + |0\rangle = |v\rangle & (\alpha\beta)|v\rangle = \alpha(\beta|v\rangle) \\ \text{each } |v\rangle \text{ has } -|v\rangle \text{ with } |v\rangle + (-|v\rangle) = |0\rangle & 1 \cdot |v\rangle = |v\rangle \end{array}$$

Structures Nest

Look at the left-hand column of that table. Those are exactly the group axioms. $(V, +)$ is an **abelian group**. A vector space is a group with extra equipment bolted on. This is why we did Section 3 first.

On Notation

From here on we write vectors as $|\psi\rangle$, read “ket psi”. For today, treat $|\cdot\rangle$ as nothing more than a decorated name for a column vector. The full Dirac formalism, including bras as dual vectors and outer products as operators, is the subject of the next class and is worth doing properly rather than in passing.

4.1 The Space We Actually Care About

Take $V = \mathbb{C}^2$ with the two distinguished vectors

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

A general element is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}.$$

Classical bit: an element of the set $\{0, 1\}$. Two options.

Qubit: a vector in the space $\mathbb{C}^2$. A continuum of options.

Two Errors to Avoid Permanently

(i) $|0\rangle$ is *not* the number zero, and it is *not* the zero vector. It is a basis vector of unit length. The zero vector is not a valid quantum state at all, since states must be normalised.

(ii) For n qubits the dimension is 2^n , not n . Two qubits live in $\mathbb{C}^4$, ten qubits in $\mathbb{C}^{1024}$. Confusing the qubit count with the dimension will wreck every tensor product calculation in Unit 3.

4.2 Span, Independence, Basis, Dimension

Span, Linear Independence, Basis

Let $\{|v_1\rangle, \dots, |v_k\rangle\} \subseteq V$.

Span. $\text{span}\{|v_i\rangle\} = \{\sum_i c_i |v_i\rangle : c_i \in \mathbb{C}\}$, the set of all linear combinations.

Linear independence. The set is linearly independent if

$$\sum_i c_i |v_i\rangle = |0\rangle \implies c_1 = c_2 = \dots = c_k = 0.$$

Basis. A linearly independent set that spans V . Every vector then has a *unique* expansion in that basis.

Dimension. The number of elements in any basis. This number does not depend on which basis you chose.

For $\mathbb{C}^2$, the set $\{|0\rangle, |1\rangle\}$ is a basis, so $\dim_{\mathbb{C}} \mathbb{C}^2 = 2$. It is called the **computational basis**. It is not the only basis, and choosing a different one is exactly what “measuring in a different basis” will mean later.

4.3 Inner Product

Inner Product and Norm

For $|u\rangle, |v\rangle \in \mathbb{C}^n$ with components u_i, v_i , define

$$\langle u | v \rangle = \sum_{i=1}^n \overline{u_i} v_i.$$

Properties:

- $\langle u | v \rangle = \overline{\langle v | u \rangle}$ (conjugate symmetry)
- linear in the second argument, conjugate-linear in the first
- $\langle v | v \rangle \geq 0$, with equality if and only if $|v\rangle = |0\rangle$

The **norm** is $\| |v\rangle \| = \sqrt{\langle v | v \rangle}$. A set is **orthonormal** if $\langle v_i | v_j \rangle = \delta_{ij}$.

Convention Clash, Be Warned Now

Physicists put the complex conjugate on the *first* argument, as above. Most pure mathematics texts put it on the *second*. Both are self-consistent and they disagree with each other. Your linear algebra textbook and your quantum textbook may well use opposite conventions. We use the physics convention throughout this course.

Check: the Computational Basis Is Orthonormal

$$\langle 0 | 0 \rangle = \bar{1} \cdot 1 + \bar{0} \cdot 0 = 1, \quad \langle 1 | 1 \rangle = 1, \quad \langle 0 | 1 \rangle = \bar{1} \cdot 0 + \bar{0} \cdot 1 = 0.$$

So $\langle i | j \rangle = \delta_{ij}$ as required.

The reason the norm matters: we will shortly require every physical state to satisfy $\| |\psi\rangle \| = 1$, so that $|\alpha|^2 + |\beta|^2 = 1$ can be read as a probability. The operations we are allowed to call quantum gates are then precisely those that preserve this norm. That constraint is what we cash in next.

Wisdom from the Back Row

“Normalisation is just quantum mechanics insisting that probabilities add up to 100%, the same way your professor insists attendance adds up to enrollment.”

5 The Hierarchy of Spaces

We have just equipped $\mathbb{C}^2$ with an inner product, almost without comment. It is worth stopping to ask what we actually bought, because the answer organises a large part of modern mathematics into a single ladder.

The Question This Section Answers

Quantum states live in a *Hilbert space*. That name is a stack of four separate structural decisions, each one adding a capability the previous layer lacked. This section unpacks the stack from the outside in, so that when you hear “Hilbert space” you hear a specification rather than a magic word.

5.1 The Four Layers

Topological Space

A **topological space** is a pair (X, τ) , where X is a set and τ is a collection of subsets of X (called **open sets**) satisfying:

T1. $\emptyset \in \tau$ and $X \in \tau$.

T2. Any union of members of τ is in τ .

T3. Any *finite* intersection of members of τ is in τ .

What it buys you: the words *near*, *limit*, and *continuous* become meaningful. Nothing else. There is no notion of how far apart two points are, no notion of length, no notion of angle.

Metric Space

A **metric space** is a pair (X, d) with $d : X \times X \rightarrow \mathbb{R}$ satisfying, for all x, y, z :

M1. $d(x, y) \geq 0$, with $d(x, y) = 0$ if and only if $x = y$.

M2. $d(x, y) = d(y, x)$.

M3. $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

What it buys you: a *number* for how far apart two points are. Every metric induces a topology, whose open sets are generated by the open balls $B_r(x) = \{y : d(x, y) < r\}$. So every metric space is a topological space.

Normed Vector Space

A **norm** on a vector space V over $\mathbb{C}$ is a map $\|\cdot\| : V \rightarrow \mathbb{R}$ with:

- N1.** $\| |v\rangle \| \geq 0$, with equality if and only if $|v\rangle = |0\rangle$.
- N2.** $\|\alpha |v\rangle \| = |\alpha| \| |v\rangle \|$ for all $\alpha \in \mathbb{C}$ (absolute homogeneity).
- N3.** $\| |u\rangle + |v\rangle \| \leq \| |u\rangle \| + \| |v\rangle \|$ (triangle inequality).

What it buys you: *length* of a vector, compatible with the linear structure. Every norm induces a metric by $d(|u\rangle, |v\rangle) = \| |u\rangle - |v\rangle \|$. So every normed space is a metric space.

Inner Product Space

An inner product $\langle \cdot | \cdot \rangle$ on V , as defined in Section 4.4.

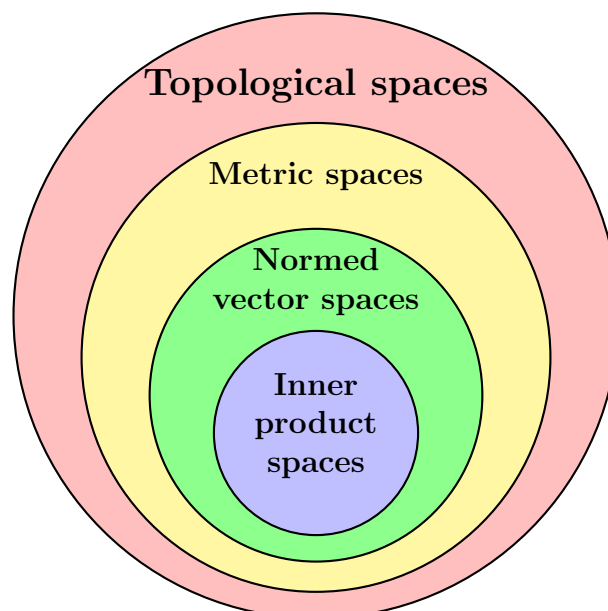
What it buys you: *angle*, and therefore **orthogonality**. Every inner product induces a norm by

$$\| |v\rangle \| = \sqrt{\langle v | v \rangle}.$$

So every inner product space is a normed space. That the induced norm really satisfies the triangle inequality is not obvious; it follows from the **Cauchy–Schwarz inequality**

$$|\langle u | v \rangle| \leq \| |u\rangle \| \| |v\rangle \|.$$

5.2 The Picture



Layer	New structure	New question you can ask
Topological	open sets	Does this sequence converge?
Metric	$d(x, y)$	How far apart are these two points?
Normed	$\ v\ $	How long is this vector?
Inner product	$\langle u \rangle v$	What is the angle between these vectors?

5.3 Every Inclusion Is Strict

Each arrow above goes one way only. Worth seeing why, because it shows the layers are genuinely different rather than notational.

A Metric That Comes From No Norm

On any vector space V define the **discrete metric**

$$d(|u\rangle, |v\rangle) = \begin{cases} 0 & |u\rangle = |v\rangle \\ 1 & \text{otherwise.} \end{cases}$$

This satisfies M1–M3, so it is a metric. But no norm induces it. If it did we would need $d(|0\rangle, 2|v\rangle) = \|2|v\rangle\| = 2\| |v\rangle \| = 2d(|0\rangle, |v\rangle)$, whereas the discrete metric gives $1 = 1$. Absolute homogeneity fails, so this metric space is not a normed space.

A Norm That Comes From No Inner Product

On $\mathbb{C}^2$ take the **supremum norm** $\|(a, b)\|_\infty = \max(|a|, |b|)$. This satisfies N1–N3. But it comes from no inner product, and there is a clean test for this.

Parallelogram law. A norm arises from an inner product if and only if

$$\| |u\rangle + |v\rangle \|^2 + \| |u\rangle - |v\rangle \|^2 = 2\| |u\rangle \|^2 + 2\| |v\rangle \|^2$$

for all $|u\rangle, |v\rangle$. (Jordan and von Neumann, 1935. The inner product is then recovered from the norm by a polarisation identity.)

Test it. Put $|u\rangle = (1, 0)$ and $|v\rangle = (0, 1)$.

Euclidean norm: $\|u + v\|^2 = 2$, $\|u - v\|^2 = 2$, so the left side is 4. The right side is $2(1) + 2(1) = 4$. Passes.

Supremum norm: $\|u + v\|_\infty = 1$ and $\|u - v\|_\infty = 1$, so the left side is 2. The right side is $2(1) + 2(1) = 4$. $2 \neq 4$, fails.

So $\|\cdot\|_\infty$ measures length perfectly well but supports no notion of angle, and hence no notion of orthogonality.

Why That Last Example Is Not a Curiosity

Orthogonality is what makes quantum measurement work. Two states are perfectly distinguishable by a single measurement precisely when $\langle \phi | \psi \rangle = 0$. Measurement bases are orthonormal bases. The Born rule is $|\langle \phi | \psi \rangle|^2$.

None of that would be available in a merely normed space. The inner product, the innermost and most demanding layer of the hierarchy, is exactly the structure quantum mechanics cannot do without.

5.4 Completeness and Hilbert Space

One more condition sits on top of the hierarchy, and it is not a fifth ring in the diagram, because it cuts across the layers rather than nesting inside them.

Cauchy Sequence, Completeness, Hilbert Space

A sequence $|v_1\rangle, |v_2\rangle, \dots$ in a metric space is **Cauchy** if its terms eventually get arbitrarily close to *each other*: for every $\epsilon > 0$ there is N with $d(|v_m\rangle, |v_n\rangle) < \epsilon$ whenever $m, n > N$.

The space is **complete** if every Cauchy sequence converges to a limit *that lies in the space*.

- A complete *normed* space is called a **Banach space**.
- A complete *inner product* space is called a **Hilbert space**.

What Incompleteness Looks Like

Take $\mathbb{Q}$ with $d(x, y) = |x - y|$. The sequence $1, 1.4, 1.41, 1.414, \dots$ is Cauchy: its terms bunch together. But its limit is $\sqrt{2}$, which is not rational. The sequence has nowhere to land *inside the space*. $\mathbb{Q}$ is incomplete; $\mathbb{R}$ is its completion.

Completeness is the guarantee that limits you are entitled to expect actually exist.

The Payoff, and an Honest Caveat

The definition you came for:

Hilbert space = complete inner product space

and the state space of a quantum system is a Hilbert space.

The caveat. In *finite* dimensions, completeness is automatic. Every finite-dimensional normed space over $\mathbb{C}$ is complete, and all norms on it are equivalent. So $\mathbb{C}^2$, and $\mathbb{C}^{2^n}$ for any n , is a Hilbert space for free, and nothing in this course will ever fail because of an incomplete space.

Why we teach it anyway. The moment you leave finite dimensions the condition has teeth:

- Position and momentum of a particle live in $L^2(\mathbb{R})$, an infinite-dimensional Hilbert space, and its completeness is a theorem (Riesz–Fischer), not a triviality.
- Photon number states span an infinite-dimensional Fock space, which is where the photonic implementations in Unit 3 actually live.
- The spectral theorem, which underwrites the entire theory of quantum measurement, needs completeness in infinite dimensions.

You are learning the general statement so that the finite-dimensional case reads as a special case rather than as the whole story.

Calibrate Your Effort Here

Do not go down the topology rabbit hole for this course. You will never need compactness, connectedness, Hausdorff separation, quotient topologies, or homeomorphisms in UE23EC343AB3.

What you *do* need is the shape of the ladder and the ability to say which layer a given piece of quantum formalism relies on. Learn the hierarchy; leave point-set topology to a topology course.

5.5 Where Each Layer Shows Up in Quantum Mechanics

Structure	What it underwrites
Inner product	Born rule $ \langle \phi \psi \rangle ^2$; orthogonality as perfect distinguishability; measurement bases; expectation values $\langle \psi A \psi \rangle$
Norm	Normalisation $\ \psi\rangle \ = 1$; probability conservation; the definition of a unitary as a norm-preserving map
Metric	Fidelity and trace distance; “how close is the state I prepared to the state I wanted”; error thresholds in fault tolerance
Completeness	Existence of limits and of spectral decompositions in infinite dimensions; L^2 , Fock space, continuous-variable quantum optics

6 From Linear Maps to Matrices

Linear Map

$T : V \rightarrow W$ is **linear** if

$$T(\alpha |u\rangle + \beta |v\rangle) = \alpha T|u\rangle + \beta T|v\rangle$$

for all vectors and all scalars.

Proposition 6.1. A linear map is completely determined by its action on a basis.

Proof. Let $\{|v_j\rangle\}$ be a basis of V and let $|x\rangle = \sum_j c_j |v_j\rangle$ be arbitrary. Then by linearity $T|x\rangle = \sum_j c_j T|v_j\rangle$, which is determined by the values $T|v_j\rangle$ alone. $\square$

So fix a basis $\{|w_i\rangle\}$ of W and expand each image:

$$T|v_j\rangle = \sum_i A_{ij} |w_i\rangle.$$

The array of coefficients A_{ij} is the **matrix of T** in these bases.

The Point of This Section

Matrices are not fundamental objects. A matrix is a bookkeeping device that records what a linear map does to a chosen basis. Change the basis and the matrix changes, while the map does not. Keeping this straight is the difference between doing quantum mechanics and shuffling symbols.

6.1 Closing the Loop

Unitary Matrix

$U \in \mathbb{C}^{n \times n}$ is **unitary** if

$$U^\dagger U = U U^\dagger = I, \quad \text{equivalently} \quad U^{-1} = U^\dagger,$$

where $U^\dagger = (U^*)^T$ is the conjugate transpose. Unitary matrices are exactly the linear maps preserving the inner product, hence the norm:

$$\langle Uu | Uv \rangle = \langle u | v \rangle \quad \text{for all } |u\rangle, |v\rangle.$$

We can now state both halves of the opening sentence in the same language.

Reversible classical gate on n bits	Quantum gate on n qubits
a bijection $B^n \rightarrow B^n$	a norm-preserving invertible linear map on $\mathbb{C}^{2^n}$
a permutation of 2^n strings	
a $2^n \times 2^n$ permutation matrix	a $2^n \times 2^n$ unitary matrix

Every permutation matrix has orthonormal columns, so every permutation matrix is unitary. Therefore

$$\{\text{permutation matrices}\} \subset \{\text{unitary matrices}\} = U(2^n)$$

and the inclusion is strict, since for instance the Hadamard matrix is unitary but is not a permutation matrix.

What We Have Actually Proved

Every reversible classical computation is a special case of a quantum computation. A quantum computer can do everything a reversible classical computer can do, and the set of unitaries is strictly larger.

Everything remaining in this course, superposition, interference, entanglement, and the speedups of Shor and Grover, lives in the gap between those two sets.

A Fortune Cookie for Unit 1

“The classical world is the special case where nobody checked the phases.”

7 Problem Set

Problems for Next Class

1. Verify that $U(2) = \{U \in \mathbb{C}^{2 \times 2} : U^\dagger U = I\}$ is a group under matrix multiplication. Check all four axioms explicitly. (Hint for closure: use $(AB)^\dagger = B^\dagger A^\dagger$.)
2. Compute all products XY, YX, YZ, ZY, ZX, XZ and write each in the form $\pm i^k P$ for a Pauli P . From your table, deduce the smallest set of phases needed to close $\{I, X, Y, Z\}$ into a group.
3. Write the 4×4 matrix of CNOT in the ordered basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$. Verify that it is simultaneously a permutation matrix and a unitary matrix.
4. Show that $HXH = Z$ and $HZH = X$. Interpret this geometrically: what is H doing to the two bases?
5. Compute $\text{ord}(S)$ and $\text{ord}(T)$ directly from the matrices, and verify $T^2 = S$ and $S^2 = Z$.
6. Is $\{I, X\}$ a group under matrix multiplication? Is $\{I, X, Y, Z\}$? Justify both answers using the axioms.
7. Show that the set of $n \times n$ permutation matrices forms a group under multiplication, and that this group has $n!$ elements.
8. Verify that the ℓ^1 norm on $\mathbb{C}^2$, defined by $\|(a, b)\|_1 = |a| + |b|$, satisfies the norm axioms N1–N3 but fails the parallelogram law. Conclude that it comes from no inner product.
9. Show directly from the axioms that the Euclidean norm on $\mathbb{C}^n$ satisfies the parallelogram law, by expanding both sides using $\| |v\rangle \|^2 = \langle v | v \rangle$. Note where you use conjugate symmetry.
10. For each of the following, state the innermost layer of the hierarchy it requires: (a) the statement $\| |\psi\rangle \| = 1$; (b) the statement $\langle 0 | 1 = 0$; (c) the statement that a sequence of states converges; (d) the statement that two density matrices are within trace distance ϵ .
11. **(Optional, harder.)** Prove that if U is unitary then every eigenvalue of U has modulus 1. Prove that if H is Hermitian ($H^\dagger = H$) then every eigenvalue of H is real.

Optional Computational Exercise (numpy only, no Qiskit needed)

```
import numpy as np
from itertools import product

I = np.eye(2, dtype=complex)
X = np.array([[0, 1], [1, 0]], dtype=complex)
Y = np.array([[0, -1j], [1j, 0]], dtype=complex)
Z = np.array([[1, 0], [0, -1]], dtype=complex)

paulis = {"I": I, "X": X, "Y": Y, "Z": Z}
phases = {1: "+", -1: "-", 1j: "+i", -1j: "-i"}

def identify(M):
    for ph, label in phases.items():
        for name, P in paulis.items():
            if np.allclose(M, ph * P):
                return f"{label}{name}"
    return "not in the group"

for (a, A), (b, B) in product(paulis.items(), repeat=2):
    print(f"{a}{b} = {identify(A @ B)}")
```

Two things to notice in the output. First, no product ever escapes the set: closure is visible rather than merely asserted. Second, the phases are unavoidable, which is precisely what Problem 2 asks you to establish by hand.

8 Reading and What Comes Next

Reading

- Nielsen & Chuang, *Quantum Computation and Quantum Information*, §2.1. Short, clean, and the standard reference for everything in Sections 4 and 5 above.
- Hidary, *Quantum Computing: An Applied Approach*, Chapter 2.
- Optional group theory background: Artin, *Algebra*, Chapter 2, §1–4 only. Stop there.

Next class

Dirac notation done properly: bras as dual vectors, outer products $|\psi\rangle\langle\phi|$ as operators, projectors, and the completeness relation. Then Hermitian and unitary operators, eigenvalues and spectral decomposition, and the postulates of quantum mechanics.

Part II

Laboratory Setup: Qiskit and Cirq

Read This First

By the end of this lab you will have **two** working Python environments, one for Qiskit and one for Cirq, and you will have run a script in each that verifies your installation by *checking the mathematics you proved by hand in Class 01*.

Setup labs are usually dead time. This one is not. The verification scripts confirm that the Pauli group closes, that $HXH = Z$, that CNOT is a permutation matrix, and that a Bell circuit produces only $|00\rangle$ and $|11\rangle$. If your environment is broken, those checks fail. If it works, you have also just seen your first four theorems confirmed numerically.

Budget: 40 minutes. Come with Python already installed if you can.

9 What We Are Installing, and Why Two of Them

Package	Version used	Role in this course
Python	3.12	The one version both SDKs are happy with
qiskit	2.5.1	Circuit construction, transpilation, operators
qiskit-aer	0.17.2	High-performance local simulator
cirq-core	1.7.0	Google's circuit framework and simulator
numpy	pulled in automatically	Everything underneath

Why Python 3.12 Specifically

Qiskit currently requires Python ≥ 3.10 . Cirq requires Python ≥ 3.11 . Both support up to 3.14.

The intersection is 3.11 and above, and 3.12 is the sweet spot: new enough for Cirq, old enough that every scientific package you will need has a prebuilt wheel for it. Choosing 3.14 will eventually work but you will be the one waiting on wheels to be published. Choosing 3.10 breaks Cirq outright.

If your system Python is 3.10 or older, install 3.12. Do not fight it.

Why Two Separate Virtual Environments

You *can* put Qiskit and Cirq in the same environment, and it will usually work. We are not going to, for one reason: when it does not work, the failure is a dependency-resolution error deep in a shared transitive dependency, which is painful to debug. Two environments, one job each. If one breaks, delete the folder and rebuild it in ninety seconds without touching the other. This costs you about 400 MB of disk and saves an unbounded amount of lab time.

10 Part A: Install Python 3.12

Check what you already have:

```
python3 --version
```

If it reports 3.11, 3.12, or 3.13, skip ahead to Section 3.

Windows

Download the 3.12 installer from <https://www.python.org/downloads/>. During installation, tick “**Add python.exe to PATH**” on the first screen. This single checkbox is responsible for most of the setup failures in this lab.

Verify in a *new* PowerShell window:

```
py -3.12 --version
```

macOS

```
brew install python@3.12
python3.12 --version
```

If you do not have Homebrew, install it from <https://brew.sh> first, or use the installer from python.org.

Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install python3.12 python3.12-venv python3-pip
python3.12 --version
```

The `python3.12-venv` package is separate on Debian-family systems and is easy to forget. Without it, virtual environment creation fails with a confusing message.

11 Part B: Build the Two Environments

Make a folder for the course, then create both environments inside it.

macOS and Linux

```
mkdir -p ~/qeqc && cd ~/qeqc

# environment 1: Qiskit
python3.12 -m venv qiskit-env
source qiskit-env/bin/activate
pip install --upgrade pip
pip install qiskit==2.5.1 qiskit-aer==0.17.2 matplotlib jupyterlab
pylatexenc
deactivate

# environment 2: Cirq
python3.12 -m venv cirq-env
source cirq-env/bin/activate
pip install --upgrade pip
pip install cirq-core==1.7.0 matplotlib jupyterlab pylatexenc
deactivate
```

Windows (PowerShell)

```
mkdir qeqc; cd qeqc

py -3.12 -m venv qiskit-env
.\qiskit-env\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install qiskit==2.5.1 qiskit-aer==0.17.2 matplotlib jupyterlab
pylatexenc
deactivate

py -3.12 -m venv cirq-env
.\cirq-env\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install cirq-core==1.7.0 matplotlib jupyterlab pylatexenc
deactivate
```

If PowerShell refuses to run the activation script, run this once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

How to Tell Which Environment You Are In

Your shell prompt gains a prefix, for example (qiskit-env) \$. If you do not see a prefix, you are in your system Python and your `import` will fail. This accounts for roughly half of all “it says module not found” reports.

Activate before every lab session. Deactivate with `deactivate`.

Why We Pin Exact Versions

Both SDKs move fast. Qiskit 1.0 was a hard break from 0.x, and Qiskit 2.0 removed a large amount of deprecated API. Tutorials you find online are frequently written against versions that no longer exist.

Pinning means every student in this lab gets identical behaviour, and errors become reproducible. If you want to experiment with newer versions, do it in a third environment, not this one.

12 Part C: Verify with the Notebook

There are no separate verification scripts. Everything lives in one notebook, `QEQC_Class01_Lab00.ipynb`, which is also the notebook we teach Part I from. Put it in your `qeqc` folder and launch it.

```
cd ~/qeqc
source qiskit-env/bin/activate # Windows: .\qiskit-env\Scripts\Activate.ps1
jupyter lab
```

Open the notebook and run every cell in order, top to bottom.

The Notebook Adapts to Your Environment

Cell 0 probes what is importable and sets `HAS_QISKIT`, `HAS_AER` and `HAS_CIRQ`. Because you built two separate environments, only one SDK is visible at a time, and the cells for the missing one print a one-line skip instead of raising `ImportError`.

Sections 1–5 use `numpy` only and run anywhere, so a broken SDK install never blocks the mathematics.

Run the notebook twice, once in each environment. The sign-off cell tells you when to switch.

12.1 The Sign-Off Cell

Every check in the notebook records its result. Section 10 collects them and prints a verdict. Screenshot it for your lab record.

```
=====
UE23EC343AB3 -- LAB 00 SIGN-OFF
=====
Python 3.12.3 | Qiskit yes | Aer yes | Cirq no
-----
[ PASS] cardinality of B^n is 2^n
[ PASS] AND is NOT reversible
[ PASS] CNOT IS reversible
[EXPECTED FAIL] {I,X,Y,Z} is closed WITHOUT phases
[ PASS] adding {+1,-1,+i,-i} closes the group
[ PASS] X and Z do NOT commute
[ PASS] X and Z anticommute (XZ = -ZX)
[ PASS] T^2 = S
```



```
[ PASS] S^2 = Z
[ PASS] only the Euclidean norm admits an inner product
[ PASS] {|0>,|1>} is orthonormal
[ PASS] CNOT is a permutation matrix
[ PASS] H is unitary but NOT a permutation
[ PASS] HXH = Z
[ PASS] Bell statevector is (|00>+|11>)/sqrt(2)
[ PASS] transpiled circuit uses only the target basis
[ PASS] only 00 and 11 appear
[ PASS] 40-qubit GHZ gives only all-zeros and all-ones
```

```
-----
28 checks run, 0 genuine failure(s)
```

```
ALL CHECKS PASSED. Environment signed off.
```

```
Now switch to the other environment and run this notebook again.
```

Shot counts will differ slightly between runs. That is sampling noise, not a fault.

One Check Is Supposed to Fail

```
[EXPECTED FAIL] {I,X,Y,Z} is closed WITHOUT phases
```

That is not a broken install. It is the lesson. The notebook multiplies every pair of Paulis and finds that $XZ = -iY$, which is *not* one of the four matrices. The set is not closed, so it is not a group, and the repair, admitting $\{\pm 1, \pm i\}$ to reach a group of order 16, is forced rather than chosen.

The sign-off cell knows about this one and does not count it against you. Any *other* FAIL is a real problem.

What to Look at While the Room Is Running Cells

Three outputs are worth stopping on.

The Pauli table (Section 2). Read down the X row: $+X$, $+I$, $+iZ$, $-iY$. Two entries carry phases that were nowhere in the original matrices. The closure failure happens in front of you.

The shot-noise table (Section 7). The deviation of $\hat{P}(00)$ from 0.5 shrinks like $1/\sqrt{N}$. Quadrupling the shots halves the error. This is why every hardware result carries error bars.

The noise sweep (Section 7). As the depolarizing probability rises, counts appear in the $|01\rangle$ and $|10\rangle$ columns. Those outcomes are *impossible* for a perfect Bell state. Every count there is decoherence, and that is the whole engineering problem of the field.

12.2 What Is in the Notebook

Section	Contents
0	Environment probe and version report
1	$ B^n = 2^n$; AND is not a bijection, CNOT is
2	Full Pauli multiplication table; closure failure; $XZ = -ZX$; orders of X, Z, H, S, T
3	Parallelogram law on the ℓ^2 , ℓ^∞ and ℓ^1 norms; incompleteness of $\mathbb{Q}$
4	Orthonormality and conjugate symmetry of the inner product
5	Unitarity and permutation tests; $HXH = Z$, $HZH = X$
6	Qiskit circuits, <code>Statevector</code> , <code>Operator</code> , transpilation to a basis gate set
7	Aer sampling, shot-noise scaling, depolarizing noise models, histograms
8	Aer simulation methods; a 40-qubit GHZ state via the stabilizer method
9	Cirq, and a side-by-side demonstration of the endianness trap
10	Lab sign-off
11	Thirteen exercises, including two computational ones

Use It in the Right Order

Work the exercises by hand *first*. The notebook is for confirming what you derived, not for discovering it. A student who runs `identify(X @ Z)` before multiplying the matrices on paper has learned to call a function, not why the Pauli group needs its phases.

13 The Same Circuit in Both SDKs

Here is a Bell state, written twice. Read the two columns against each other; the differences are almost entirely cosmetic.

Qiskit	Cirq
<code>from qiskit import QuantumCircuit</code>	<code>import cirq</code>
qubits are integer indices	qubits are <i>objects</i> : <code>cirq.LineQubit(0)</code>
<code>qc = QuantumCircuit(2, 2)</code>	<code>q0, q1 = cirq.LineQubit.range(2)</code>
<code>qc.h(0)</code>	<code>cirq.H(q0)</code>
<code>qc.cx(0, 1)</code>	<code>cirq.CNOT(q0, q1)</code>
<code>qc.measure([0,1], [0,1])</code>	<code>cirq.measure(q0, q1, key="m")</code>
<code>AerSimulator().run(...)</code>	<code>cirq.Simulator().run(...)</code>
<code>shots=4000</code>	<code>repetitions=4000</code>

The One Real Conceptual Difference

In Qiskit, a qubit is an index into a register you declared up front. In Cirq, a qubit is a first-class object with an identity and often a physical location, such as `cirq.GridQubit(3, 5)`.

That is not decoration. Cirq was built around Google's superconducting chips, where which qubit is adjacent to which is a hard constraint on what circuits you can run. The qubit-as-object design makes connectivity a visible part of your program rather than something the transpiler hides from you.

Bit Ordering: The Bug That Will Get You

Qiskit and Cirq disagree about which end of a bitstring is which.

- **Qiskit** is **little-endian**: in the string '01', the *rightmost* character is qubit 0.
- **Cirq** is **big-endian**: the *leftmost* character is the first qubit you listed.

For the Bell state this is invisible, because $|00\rangle$ and $|11\rangle$ are palindromes. The moment you build a circuit with an asymmetric outcome, the two SDKs will appear to disagree and you will spend an hour hunting a bug that does not exist.

Both are correct. They are the same physics in opposite notation. Write it on the inside cover of your lab notebook.

Overheard in Office Hours, Part 2

"Endianness is the one place where computer science makes you feel the same pain as a British vs. American spelling argument, except getting it wrong actually crashes your simulation."

14 Optional: Real Hardware via IBM Quantum

Everything in Units 1 and 2 runs on your laptop. Hardware access is optional and can wait, but the account takes a few minutes to create and is worth doing early.

IBM Changed Their Platform. Old Tutorials Are Wrong.

IBM Quantum Platform “Classic” has been sunset. Any tutorial, textbook, or Stack Overflow answer that uses one of the following is now broken:

```
from qiskit import IBMQ # removed entirely
IBMQ.load_account() # removed entirely
QiskitRuntimeService(channel="ibm_quantum") # channel no longer valid
```

The current channel is `ibm_quantum_platform`, and the service now lives on IBM Cloud. Instances are identified by a Cloud Resource Name (CRN) rather than the old `hub/group/project` string.

Because this area is actively changing, **check the current documentation before you debug anything**: <https://quantum.cloud.ibm.com/docs>

Setting Up Hardware Access

1. Create a free Open plan account at the IBM Quantum Platform login page.
2. From the dashboard, create an API key. **It is shown once.** Copy it somewhere safe immediately.
3. Copy your instance CRN from the Instances tab of the dashboard.
4. In your `qiskit-env`:

```
pip install qiskit-ibm-runtime
```

```
from qiskit_ibm_runtime import QiskitRuntimeService

QiskitRuntimeService.save_account(
    channel="ibm_quantum_platform",
    token="PASTE_YOUR_API_KEY_HERE",
    instance="PASTE_YOUR_CRN_OR_INSTANCE_NAME",
    overwrite=True,
)

service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False,
    min_num_qubits=5)
print("least busy backend:", backend.name)
```

Never Commit Your API Key

Do not paste your token into a notebook you then push to GitHub. Bots scrape public repositories for exactly this. `save_account` writes the credential to a config file once; after that, plain `QiskitRuntimeService()` finds it with no token in your source.

Add `*.env`, `.qiskit/`, and your venv folders to `.gitignore`.

On Cirq and Hardware

Cirq's `cirq-google` module targets Google's Quantum Computing Service, which is not open to the public. For this course, Cirq is a simulator and a circuit-construction framework, and that is entirely sufficient.

If you want Cirq talking to real devices, the accessible routes are `cirq-ionq` and `cirq-aqt`, both of which need vendor accounts. Out of scope for the lab sequence, but worth knowing exists.

15 Troubleshooting

Symptom	Cause and fix
<code>ModuleNotFoundError: No module named 'qiskit'</code>	The environment is not activated. Look for the <code>(qiskit-env)</code> prefix on your prompt.
<code>python: command not found</code> (Windows)	Use <code>py -3.12</code> instead, or reinstall Python with "Add to PATH" ticked.
<code>Ensurepip is not available</code> (Linux)	Install <code>python3.12-venv</code> .
<code>cannot be loaded because running scripts is disabled</code> (Windows)	Run <code>Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned</code> once.
Cirq install fails with a Python version error	You are on 3.10 or older. Cirq requires 3.11 or newer.
<code>ImportError: cannot import name 'IBMQ'</code>	That API was removed. See Section 6.
<code>ImportError: cannot import name 'execute'</code>	Removed in Qiskit 1.0. Use <code>transpile</code> then <code>backend.run</code> , as the verification script does.
Very long build times, compiler errors	<code>pip</code> is building from source because no wheel exists for your Python version. Drop back to 3.12.
Jupyter cannot see the packages	Register the kernel: <code>python -m ipykernel install --user --name qeqc-qiskit</code>

Symptom	Cause and fix
Notebook cells print “Skipped: activate ...”	Working as designed. That SDK is in the other environment. Run the notebook again there.
<code>NameError</code> halfway down the notebook	You ran cells out of order. Kernel → Restart and Run All.
<code>qc.draw('mpl')</code> raises an error	Install <code>pylatexenc</code> . The text drawer used in the notebook needs nothing extra.
Everything is broken and you are out of time	Delete the venv folder and rebuild. It takes ninety seconds. This is why we used two.

Fallback: Google Colab

If your machine defeats you, both SDKs run in Colab with no local install:

```
!pip install qiskit qiskit-aer pylatexenc # in one copy of the notebook
!pip install cirq-core # in another copy
```

Use separate notebooks, for the same reason we used separate environments. Colab is a fine safety net, but get the local install working before the assessed labs. You will want to run things that take longer than a Colab session lasts.

16 Before You Leave

Checklist

1. The notebook’s sign-off cell prints ALL CHECKS PASSED in `qiskit-env`.
2. The notebook’s sign-off cell prints ALL CHECKS PASSED in `cirq-env`.
3. You understand why `{I,X,Y,Z}` is closed WITHOUT phases is an EXPECTED FAIL.
4. You can activate and deactivate both environments without looking up the command.
5. You have written down which SDK is little-endian and which is big-endian.
6. *(Optional)* Your IBM Quantum API key is saved and `least_busy` returns a backend name.

Last One, Promise

“A qubit walks into a bar. Or doesn’t. It’s in a superposition until the bartender looks.”

Next Lab

Building circuits from scratch in both SDKs: single-qubit gates as Bloch sphere rotations, statevector inspection, and measurement statistics. We will verify $R_y(\theta_1)R_y(\theta_2) = R_y(\theta_1 + \theta_2)$ numerically against the proof in the problem bank.

Quantum Entanglement & Quantum Computing PES University

Bring Problem 2 worked out by hand, and a working environment.